Innovators@DravidianCodeMix 2026: A Unified Transformer-Based Approach to Hate and Hope Speech Detection in Code-Mixed Dravidian Languages

Anonymous Authors

Abstract

This paper describes three systems submitted to DravidianCodeMix 2026 across three task settings: Tamil-English hate speech, Tulu hope speech, and Kannada hate speech. We develop multilingual Transformer-based classifiers using XLM-R encoders with task-specific prediction heads. The Tamil-English system uses a hierarchical classifier for five annotation levels, the Tulu system jointly predicts coarse- and fine-grained hope-speech labels, and the Kannada system performs binary hate-speech classification. The training pipeline includes supervised fine-tuning, label encoding, class-imbalance handling where applicable, AdamW optimization, learning-rate scheduling, gradient clipping, mixed-precision GPU training, early stopping, checkpointing, and a reproducible prediction pipeline. In the official DravidianCodeMix 2026 leaderboard, the submitted system ranked second in Subtask 1 with an overall Macro-F1 of 0.4394, first in Subtask 2 with a Macro-F1 of 0.4348 and accuracy of 0.5121, and third in Subtask 3 with a Macro-F1 of 0.7652. These results demonstrate the competitiveness of adapting a common multilingual Transformer foundation to different annotation structures and linguistic settings. Model selection for Subtask 1 was performed using Level-4 validation Macro-F1. After every training epoch, predictions were collected over the complete validation set and Macro-F1 was calculated independently for all five levels. The checkpoint achieving the highest Level-4 Macro-F1 was retained, with early stopping using a patience of three epochs to limit over-training.

Keywords

hate speech detection, hope speech detection, code-mixed text, Dravidian languages, multilingual Transformers, XLM-R, hierarchical classification, multi-task learning

1. Introduction

Online communication in Dravidian languages frequently combines native scripts, English, transliterated text, informal spelling, emojis, abbreviations, and context-dependent expressions. These characteristics make automatic detection of harmful and supportive speech substantially more difficult than conventional monolingual text classification. The same semantic expression can appear in multiple scripts or spelling variants, while short social-media comments may provide limited context.

DravidianCodeMix 2026 provides a shared evaluation setting for hate and hope speech detection across Dravidian languages [1, 2]. The study addresses three task settings: Tamil-English hate speech, Tulu hope speech, and Kannada hate speech. We use a common multilingual Transformer foundation while adapting the output architecture to each task: hierarchical prediction for Tamil-English, joint coarse/fine prediction for Tulu, and binary classification for Kannada.

The main contributions of this paper are: (1) a unified multilingual Transformer-oriented workflow covering all three subtasks; (2) task-specific classification heads adapted to different label structures; (3) class-imbalance-aware training for the Kannada task; (4) validation using accuracy, macro-F1, weighted-F1, and the task-specific joint score; (5) systematic debugging of training and prediction behavior; and (6) a reproducible model-checkpoint and submission-generation pipeline.

Forum for Information Retrieval Evaluation (FIRE 2026), December 17–20, 2026, India



© 2026 This work is licensed under a "CC BY 4.0" license.

Table 1
Tamil-English hierarchical classifier input dimensions and output classes.

Level	Official annotation	Input dimension	Output classes
L1	Class	1024	3
L2	Target	1027	6
L3	Category	1030	3
L4	Subcategory	1027	32
L5	Bias	1056	7

2. Task Description

2.1. Tamil-English Hate Speech

The five levels are not independent classifiers. Let $h \in \mathbb{R}^{1024}$ denote the shared Transformer representation, and let z_i denote the logits produced by level i. The classification heads are computed sequentially:

$$
\begin{aligned}
z_1 &= f_1(h), \\
z_2 &= f_2([h; z_1]), \\
z_3 &= f_3([h; z_2]), \\
z_4 &= f_4([h; z_3]), \\
z_5 &= f_5([h; z_4]).
\end{aligned}
$$

Here, $[\cdot;\cdot]$ denotes vector concatenation. Only the raw logits from the immediately preceding level are passed to the next head; predicted class labels and logits from non-adjacent levels are not used as inputs.

2.2. Tulu Hope Speech

The Tulu setting focuses on identifying hope or supportive speech. The system predicts both coarse-grained and fine-grained hope-speech labels using a shared multilingual representation and two task-specific classification heads. The development data used in the reported run contains 3,513 training samples and 753 validation samples, with three coarse classes and five fine classes.

2.3. Kannada Hate Speech

The Kannada setting focuses on binary hate speech classification. The reported experiment contains 3,040 training samples and 500 validation samples. The training labels contain 1,687 instances of label 0 and 1,353 instances of label 1, while the validation labels contain 288 instances of label 0 and 212 instances of label 1. The distribution motivates attention to balanced training and macro-F1.

3. Dataset and Preprocessing

The task datasets are loaded from CSV files and unnecessary index columns are removed when present. Labels are converted to integer identifiers using LabelEncoder objects fitted on the training labels. The fitted encoders are serialized so that integer predictions can later be converted back to the original task labels during submission generation. The broader DravidianCodeMix dataset context is described by Chakravarthi et al. [3].

Comments are converted to strings and tokenized with the tokenizer associated with the selected XLM-R model. Input sequences are truncated to the configured maximum length and padded as required by each training pipeline. Attention masks are supplied to the encoder to distinguish actual tokens from padding. The same tokenizer/model family and compatible preprocessing are used during prediction.

The preprocessing intentionally avoids aggressive normalization. This preserves script identity, transliteration patterns, punctuation, emojis, and informal spelling that may contain useful classification information. Potential improvements include transliteration-aware normalization, data augmentation, spelling normalization, and multilingual error analysis.

4. Methodology

4.1. Multilingual Transformer Encoder

The main modelling strategy uses FacebookAI/xlm-roberta-large for the Tamil-English and Tulu experiments and FacebookAI/xlm-roberta-base for the reported Kannada experiment. XLM-R produces contextual token representations that are then passed to task-specific classification components [4]. The pretrained encoders are fine-tuned rather than used only for zero-shot prediction.

4.2. Representation and Classification Head

For Tamil-English, a hierarchical classifier uses a shared encoder representation and five classification layers with output sizes of 3, 6, 3, 32, and 7. For Tulu, a shared feed-forward representation is connected to a three-class coarse head and a five-class fine head. For Kannada, the classifier uses dropout, a linear projection, ReLU, a second dropout layer, and a final two-class linear layer. Predictions are obtained from the highest output logit.

4.3. Class-Imbalance Handling

For the Tamil-English hierarchical classifier, balanced class weights were computed independently for each of the five levels and incorporated into their respective cross-entropy losses. The overall hierarchical objective combines the five losses with weights of 1.0, 1.2, 1.0, 1.8, and 1.0 for Levels 1-5, respectively. Hence, the Level-4 loss contributes more strongly to the optimization objective while all five levels remain jointly trained.

Balanced class weights are explicitly used in the Kannada training pipeline with weighted cross-entropy loss. This reduces the tendency to favor the more frequent class and makes macro-F1 a useful diagnostic for minority-class behavior. The Tulu multi-task setting optimizes the coarse and fine objectives jointly, while the Tamil-English model optimizes its hierarchical outputs.

4.4. Optimization

The models are fine-tuned using AdamW. The training pipelines use learning-rate scheduling with warm-up/decay, gradient clipping, checkpointing, and early stopping based on validation performance. The reported runs use GPU acceleration and automatic mixed precision where configured.

4.5. Mixed Precision

Automatic mixed precision was used in the training pipeline where configured to reduce GPU memory consumption and improve training efficiency. The exact precision configuration depended on the individual task's training implementation.

5. Experimental Setup

Table 2 summarizes the principal configurations used in the three subtasks. The Tamil-English system uses a hierarchical classifier with five output heads, while the Tulu system jointly predicts coarse- and fine-grained labels. The Kannada system uses XLM-R base with weighted cross-entropy for binary classification. The reported configurations reflect the documented training workflows; additional hyperparameters should be reported only where they are confirmed by the corresponding training logs.

Table 2
Experimental configuration and dataset sizes for the three reported subtasks.

Subtask	Model / Configuration
Subtask 1	XLM-R large; hierarchical heads (3/6/3/32/7); 8 epochs
Subtask 2	XLM-R large; shared representation with coarse/fine heads; 10 epochs
Subtask 3	XLM-R base; maximum length 128; batch size 8; learning rate 5×10^{-5}; weighted cross-entropy; AdamW; 10 epochs

Table 3
Official leaderboard performance across the three DravidianCodeMix 2026 subtasks.

Subtask	Language / Setting	Accuracy	Macro-F1	Official / Final
Subtask 1	Tamil-English Hate Speech	–	0.4394	Official / Rank 2
Subtask 2	Tulu Hope Speech	0.5121	0.4348	Official / Rank 1
Subtask 3	Kannada Hate Speech	–	0.7652	Official / Rank 3

Table 4
Official level-wise Macro-F1 for Subtask 1 (Tamil-English Hate Speech).

Subtask 1		L1 Macro-F1	L2 Macro-F1	L3 Macro-F1	L4 Macro-F1	L5 Macro-F1	Overall Macro-F1
Tamil-English Speech	Hate	0.5403	0.5231	0.6729	0.1048	0.3560	0.4394

Table 5
Validation-to-official performance gap for the Tamil-English hierarchical task. Values are calculated as official Macro-F1 minus best validation Macro-F1.

Level	Best validation Macro-F1	Official Macro-F1	Gap
L1	0.5468	0.5403	-0.0065
L2	0.6118	0.5231	-0.0887
L3	0.6732	0.6729	-0.0003
L4	0.1162	0.1048	-0.0114
L5	0.3440	0.3560	+0.0120

6. Results

The final results reported in this section are the team's official DravidianCodeMix 2026 leaderboard results. Development/validation results were used during model selection and debugging, whereas the values below correspond to the submitted systems evaluated by the shared-task organizers. Subtask 1 is ranked by the mean Macro-F1 across Levels 1–5; Subtask 2 reports the official Macro-F1 for the submitted run together with accuracy; and Subtask 3 is ranked by Macro-F1.

The official results differ from intermediate validation results recorded during development. In particular, an early Kannada experiment exhibited prediction collapse, favoring class 1 for most or all validation examples. This issue was investigated during debugging, and the affected run is not representative of the final submission. The submitted Kannada system achieved an official Macro-F1 of 0.7652. Accordingly, the results reported in this section refer to the official leaderboard unless explicitly identified as validation results.

The submitted system ranked second in Subtask 1 (Tamil-English hate speech), first in Subtask 2 (Tulu hope speech), and third in Subtask 3 (Kannada hate speech). In Subtask 1, the overall Macro-F1 was 0.4394, which was 0.0209 below the leading score of 0.4603 achieved by HateLens. In Subtask 2, the system achieved a Macro-F1 of 0.4348 and an accuracy of 0.5121, exceeding the second-ranked overall

score of 0.3981 by 0.0367. In Subtask 3, the system achieved a Macro-F1 of 0.7652, with precision of 0.7653 and recall of 0.7679, placing third behind TokenX (0.7720) and Coders (0.7672).

For Subtask 1, the level-wise Macro-F1 scores were 0.5403, 0.5231, 0.6729, 0.1048, and 0.3560 for Levels 1–5, respectively. These values yield an overall Macro-F1 of 0.4394 and reveal substantial variation across the hierarchical annotation levels. Although the system performed comparatively better at Levels 1–3, Level 4 was the principal weakness, with a Macro-F1 of 0.1048 compared with 0.1521 for HateLens. This result highlights the difficulty of predicting fine-grained hierarchical categories and motivates further investigation into label distributions, class confusion, and task-specific training strategies. Table 5 shows that Level 2 had the largest validation-to-official decrease, from 0.6118 to 0.5231, while Level 5 improved from 0.3440 to 0.3560.

The Tulu system jointly predicts three coarse-grained classes and five fine-grained classes using a shared XLM-R representation with separate classification heads. Its official overall Macro-F1 of 0.4348 secured first place in the subtask. For the L1 coarse-tone component, the reported Macro-F1 was 0.3019, numerically equal to MMIP Lab's score, although the published component ranking placed the submitted system second. These results demonstrate the competitiveness of the joint prediction approach while indicating that component-level performance can differ from the overall task ranking.

The Kannada system uses XLM-R base followed by a feed-forward classification head and weighted cross-entropy to address class imbalance. The development experiments exposed the importance of checking prediction distributions, label encoding, training-loop execution, and validation accumulation. In particular, an unintended early termination of training can prevent the model from receiving the intended optimization updates, while incomplete validation predictions can produce misleading metrics. The final submitted system achieved an official Macro-F1 of 0.7652 and ranked third, demonstrating the importance of validating the complete training and evaluation pipeline.

Overall, the results support using a shared multilingual Transformer foundation with task-specific prediction heads for heterogeneous annotation schemes. However, the differences across languages and tasks show that a common architecture and training strategy do not guarantee uniform performance. Future work should prioritize improving Level-4 Tamil-English classification, tuning task-specific learning rates and sequence lengths, exploring label-aware sampling and transliteration-aware preprocessing, and conducting systematic error analysis for the Kannada and Tulu settings.

7. Error Analysis and Observations

The Kannada experiments exposed an important failure mode in which the classifier initially favored one class. Logit inspection showed a consistent preference for class 1, and an early validation report produced zero precision, recall, and F1 for class 0. This behavior motivated checks of the label encoder, class distributions, optimizer updates, training-loop control flow, and prediction accumulation.

A particularly important implementation issue was an unintended break inside the training loop. Such a break terminates an epoch after a single batch and prevents the model from receiving the intended number of optimization updates. Another issue occurred when the loop was written as `for step, batch in train_loader`. A DataLoader returns each batch directly, so `enumerate(train_loader)` is required when an explicit step index is needed. Validation predictions and targets must also be accumulated over the complete validation set before metrics are computed.

The three task settings exhibit different sources of difficulty. Tamil-English contains code switching, transliteration, spelling variation, sarcasm, and implicit abuse. Tulu presents a low-resource multilingual setting with both coarse and fine-grained labels. Kannada includes short comments, lexical overlap, class imbalance, and ambiguous or context-dependent expressions. These factors can lead to confusion between harmful, neutral, and supportive language.

The observed error patterns suggest that the three subtasks require different improvements. The low Level-4 Macro-F1 in Tamil-English motivates closer examination of fine-grained label confusion and class support. For Tulu, further analysis should distinguish errors in coarse-grained and fine-grained predictions. For Kannada, prediction-distribution checks, complete validation accumulation,

and verification of training-loop execution are essential safeguards against misleading metrics. Future experiments should use per-class precision and recall, confusion matrices, and representative error examples to identify task-specific weaknesses.

8. Prediction and Submission Pipeline

After training, the best model checkpoint and label encoder(s) are retained. The prediction script loads the same tokenizer, encoder architecture, classifier structure, and saved parameters used during training. Test comments are tokenized with the same preprocessing configuration, and predictions are generated by selecting the maximum output logit for each task head.

Integer predictions are converted back to the original labels using the saved encoders. The resulting task-specific prediction files must follow the current DravidianCodeMix 2026 submission requirements for filenames, columns, delimiter, and packaging. The supplied Kannada workflow produces a task-specific prediction TSV/CSV artifact and the team's submission ZIP contains the individual language outputs required by the platform.

Before submission, the prediction pipeline should verify that the number of predictions equals the number of test instances, every required ID occurs exactly once, labels belong to the permitted label set, no training or validation rows are included, and the ZIP archive contains only the required submission files.

9. Reproducibility

The implementation uses Python, PyTorch, Hugging Face Transformers, pandas, NumPy, and scikit-learn. The reported Kannada workflow fixes the random seed at 42. Label encoders are serialized for reproducible label decoding, and the best model parameters are saved as task-specific checkpoint files. The Tamil-English and Tulu runs likewise retain their trained model/checkpoint and label-encoding artifacts.

Code repository: https://anonymous.4open.science/r/Innovators-DravidianCodeMix-2026-52B7/.

10. Conclusion

This paper presents a unified Transformer-based approach in DravidianCodeMix 2026 across Tamil-English hate speech, Tulu hope speech, and Kannada hate speech. XLM-R provides a common multilingual representation while task-specific heads accommodate hierarchical, multi-task, and binary classification settings. The official leaderboard results show that the submitted system ranked second in Subtask 1, first in Subtask 2, and third in Subtask 3, demonstrating competitive performance across all three settings. The results also highlight the difficulty of deeper hierarchical labels and the benefit of task-specific modelling for heterogeneous annotation schemes.

Future work includes transliteration-aware preprocessing, data augmentation, task-specific hyperparameter tuning, parameter-efficient fine-tuning, ensemble methods, threshold calibration, and systematic multilingual error analysis. These improvements may be particularly useful for low-resource languages, highly code-mixed comments, and examples where harmful or supportive intent is expressed indirectly.

Acknowledgments

We thank the DravidianCodeMix 2026 organizers and the FIRE community for providing the shared-task resources and evaluation framework.

References

[1] S. N, S. Rajiakodi, D. Thenmozhi, B. B, S. C N, P. K. Pannerselvam, Kathiravan an Kumaresan, A. M D, P. R Hegde, D. Vikram, R. Advanthaya, R. Shetty P, A. Hegde, H. K S, M. Subramanian, C. Rajkumar, B. R. Chakravarthi, Overview of the shared task on hate and hope speech detection in dravidian languages, in: Forum of Information Retrieval and Evaluation (FIRE 2026), 2026.

[2] DravidianCodeMix 2026 Organizers, Dravidiancodemix 2026: Shared task on hate and hope speech detection in dravidian languages, FIRE 2026, 2026. URL: https://sites.google.com/view/dravidiancodemix-2026/, accessed 9 October 2026.

[3] B. R. Chakravarthi, R. Priyadharshini, V. Muralidaran, N. Jose, S. Suryawanshi, E. Sherly, J. P. McCrae, Dravidiancodemix: Sentiment analysis and offensive language identification dataset for dravidian languages in code-mixed text, Language Resources and Evaluation 56 (2022) 765–806. doi:10.1007/s10579-022-09583-7.

[4] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, V. Stoyanov, Unsupervised cross-lingual representation learning at scale, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020, pp. 8440–8451.

[5] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, Roberta: A robustly optimized bert pretraining approach, 2019. arXiv:1907.11692.

[6] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, 2019. URL: https://openreview.net/forum?id=Bkg6RiCqY7, international Conference on Learning Representations.